12.2 I/O with File Pointers

For each of the I/O library functions we've been using so far, there's a companion function which accepts an additional file pointer argument telling it where to read from or write to. The companion function to `printf` is `fprintf`, and the file pointer argument comes first. To print a string to the `output.dat` file we opened in the previous section, we might call

```
fprintf(ofp, "Hello, world!\n");
```

The companion function to `getchar` is `getc`, and the file pointer is its only argument. To read a character from the `input.dat` file we opened in the previous section, we might call

```
int c;
c = getc(ifp);
```

The companion function to `putchar` is `putc`, and the file pointer argument comes last. To write a character to `output.dat`, we could call

```
putc(c, ofp);
```

Our own `getline` function calls `getchar` and so always reads the standard input. We could write a companion `fgetline` function which reads from an arbitrary file pointer:

```
#include <stdio.h>

/* Read one line from fp, */
/* copying it to line array (but no more than max chars). */
/* Does not place terminating \n in line array. */
/* Returns line length, or 0 for empty line, or EOF for end-of-file. */

int fgetline(FILE *fp, char line[], int max)
{
int nch = 0;
int c;
max = max - 1;                  /* leave room for '\0' */

while((c = getc(fp)) != EOF)
        {
        if(c == '\n')
                break;

        if(nch < max)
                {
                line[nch] = c;
                nch = nch + 1;
                }
        }

if(c == EOF && nch == 0)
        return EOF;

line[nch] = '\0';
return nch;
}
```

Now we could read one line from `ifp` by calling

```
char line[MAXLINE];
...
```

```
        fgetline(ifp, line, MAXLINE);
```

Read sequentially: prev next up top